

DeepOHeat-v1 sul Mac

Diario degli esperimenti: cosa abbiamo fatto girare, cosa è andato storto, cosa abbiamo scoperto.
28–29 agosto 2026.

In due parole

DeepOHeat-v1 è una rete neurale che, data la **mappa di potenza** di un chip (dove e quanto scaldano i componenti, una griglia 21×21), prevede la **temperatura in tutto il volume** del chip. È il surrogato veloce di un solutore numerico: millisecondi invece di secondi. Viene dal paper *DeepOHeat-v1: Efficient Operator Learning for Fast and Trustworthy Thermal Simulation and Optimization in 3D-IC Design* (Yu et al., UC Santa Barbara, IEEE TCPMT 2025, arXiv 2504.03955).

Due particolarità che spiegano quasi tutto quello che segue. **Uno**: la rete non impara da esempi "mappa → temperatura giusta", ma controllando che la temperatura che propone rispetti l'equazione del calore (approccio *physics-informed*). **Due**: per essere veloce ricostruisce il campo 3D come somma di prodotti di funzioni 1D (struttura "separabile"), e questo è anche il motivo per cui su una CPU portatile un intero training dura circa un minuto.

Setup e fix per farlo partire

- Ambiente: `.venv` con Python 3.12, `jax` 0.11 (solo CPU), `equinox`, `optax`. Nessun `requirements.txt` nella repo: l'abbiamo scritto noi.
- La repo **non** contiene pesi pre-allenati né soluzioni di training: solo codice + dati di input (Google Drive). Tutti i modelli qui sotto sono allenati da zero.
- Fix 1 — `jax.tree_map` rimosso nelle versioni recenti → `jax.tree_util.tree_map`.
- Fix 2 — crash nell'ottimizzatore: lo stato di Adam era inizializzato su tutti gli array del modello, i gradienti coprono solo quelli float. Le versioni nuove di `optax` non tollerano più la differenza → `optimizer.init(eqx.filter(model, eqx.is_inexact_array))`.
- Aggiunti tre flag a `heat_surface.py` (default = comportamento originale): `--decay_steps`, `--train_data`, `--tag`.
- Tempo: ~7 ms per iterazione → 70 s per 10.000 epoche, 6 min per 50.000. Il paper riporta 30 s su RTX 3090.

Esperimento 1 — Riproduzione del paper (default)

I default della repo sono esattamente il setup del paper: 10.000 epoche, Adam lr 1e-3 con decadimento ×0,9 ogni 500 passi, 50 mappe per passo. Il test set sono 10 mappe di potenza "a blocchi" con soluzione di riferimento calcolata da un solutore commerciale (Celsius 3D).

	MAPE (metrica del paper)	rel. L2	tempo
Paper (RTX 3090)	0,049%	—	30 s
Noi, default (CPU)	0,058%	2,96%	70 s

Numeri allineati al paper. Ma guardando la **prima mappa di test** si vede il problema che il MAPE nasconde: la predizione azzecca il gradiente generale ma **liscia via i punti caldi**. Il picco vero è 33,5 °C, predetto 31,5 °C. Il MAPE è calcolato in Kelvin (300 K di base) e mediato sull'intero volume: un errore locale di 2 °C ci sparisce dentro.

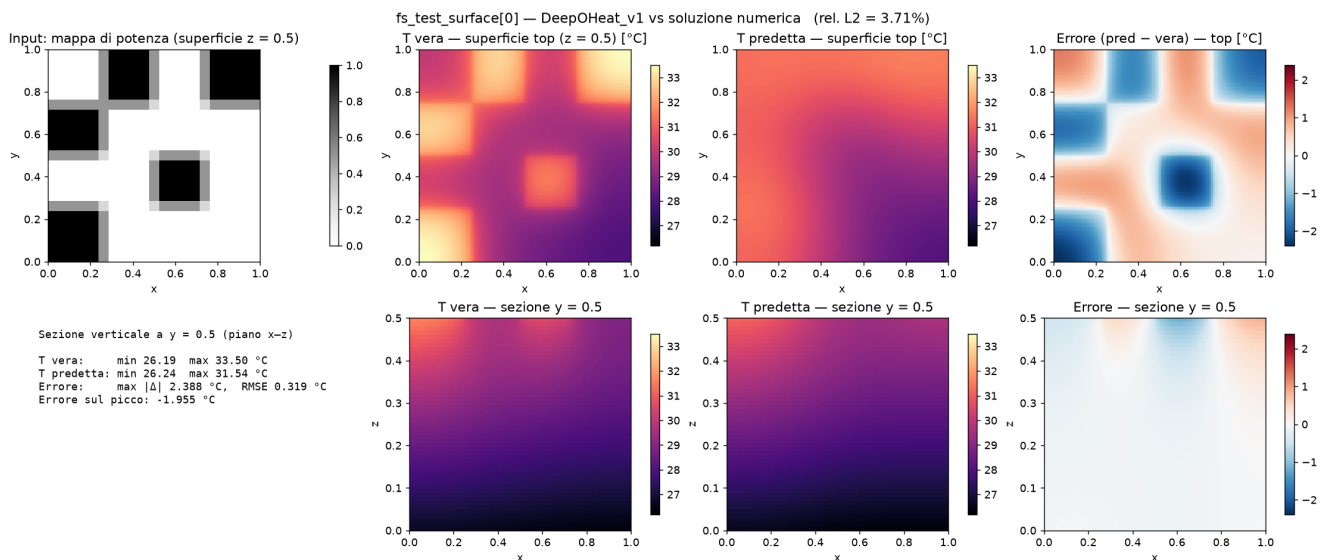


Fig. 1 — Mappa di test 0. Sopra: potenza in ingresso, temperatura vera sulla superficie superiore, predizione, errore. Sotto: sezione verticale a metà chip. La mappa d'errore ricalca la mappa di potenza: blu (sottostima) sotto i blocchi caldi.

Esperimento 2 — Più epoche (50.000)

Prima idea, la più ovvia: allenare di più. Migliora, ma poco, e il log della loss spiega perché: da 20.000 epoche in avanti è piatta. Lo scheduler porta il learning rate a $\sim 5e-6$ già a 25.000 passi — il training si spegne da solo.

finestra di epoche	loss media	learning rate a fine finestra
5k – 10k	0,167	1,2e-4
10k – 15k	0,096	4,2e-5
15k – 20k	0,079	1,5e-5
20k – 25k	0,068	5,2e-6
25k – 50k	0,067 – 0,069	< 2e-6

Morale: con lo scheduler originale, 20–25k epoche è il punto giusto; oltre è tempo sprecato.

Esperimento 3 — Decadimento del learning rate più lento

Decadimento ogni 1.000 passi invece di 500, 50.000 epoche. Il learning rate resta utile per tutto il budget. La loss finale scende di $4\times$ ($0,067 \rightarrow 0,015$) e sulla prima mappa il picco è finalmente centrato (errore $+0,1$ °C).

	10k, decay 500	50k, decay 500	50k, decay 1000
rel. L2 medio test	2,96%	2,04%	1,96%
errore sul picco (max_l1)	0,057	0,027	0,019
mappa 0: errore sul picco	-1,96 °C	-0,71 °C	+0,12 °C
mappa 0: max errore	2,39 °C	2,03 °C	1,24 °C

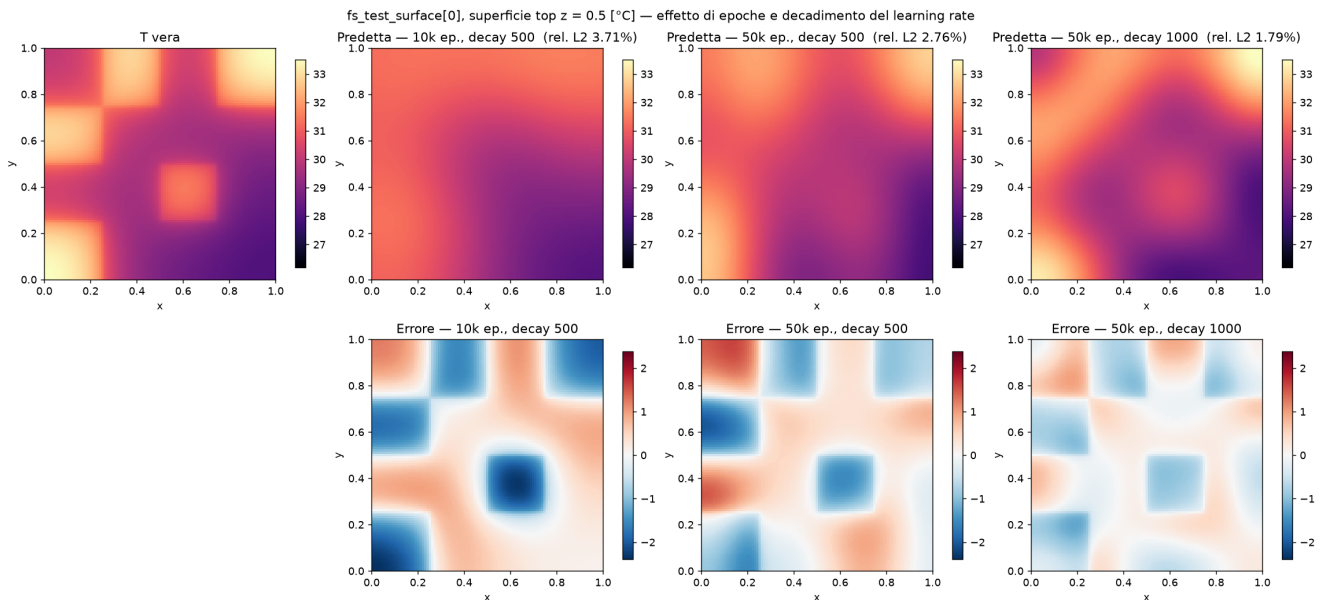


Fig. 2 — Mappa di test 0, superficie superiore. Con il decadimento più lento (ultima colonna) i blocchi diventano riconoscibili.

Un dato onesto però: la loss di training è scesa 4×, ma l'errore **medio** sul test solo da 2,04% a 1,96%. Quindi l'ottimizzazione non è più il collo di bottiglia. Il residuo è altrove.

Intermezzo — Perché le predizioni sono sfocate?

Due cause, una fisica e una di dati.

La fisica smussa (un po'). La mappa di potenza ha bordi a gradino, la temperatura no: il calore diffonde. In superficie, dove entra il calore, la transizione al bordo di un blocco si spalma su $\sim 0,2$ di dominio; scendendo nello spessore i dettagli si fondono fino a un fondo quasi uniforme. Quindi *l'obiettivo non sono quadrati netti*: è la quantità giusta di smussamento.

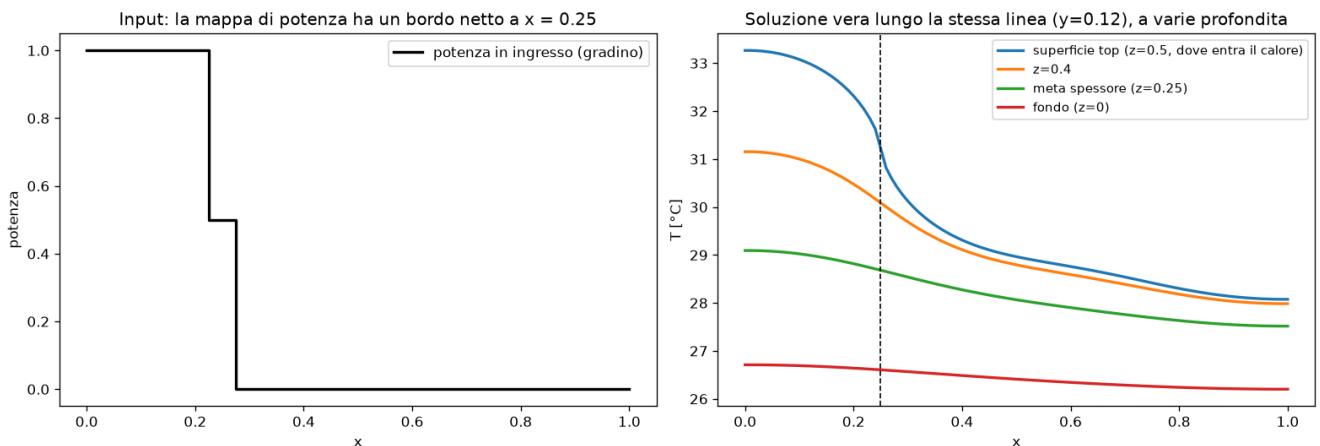


Fig. 3 — Sinistra: la potenza lungo una linea che attraversa un blocco (gradino). Destra: la temperatura vera lungo la stessa linea a varie profondità. Il "ginocchio" sul bordo è visibile solo in superficie.

I dati di training sono di un altro tipo. Il paper allena su *campi gaussiani casuali*: nuvole sfumate, con valori positivi e negativi. Le mappe di test sono *rettangoli a gradino*. La rete non ha mai visto un bordo netto in allenamento: quando ne incontra uno, lo codifica con il vocabolario delle sfumature. Il paper lo ammette a pag. 9: le mappe di test "differiscono significativamente" da quelle di training.

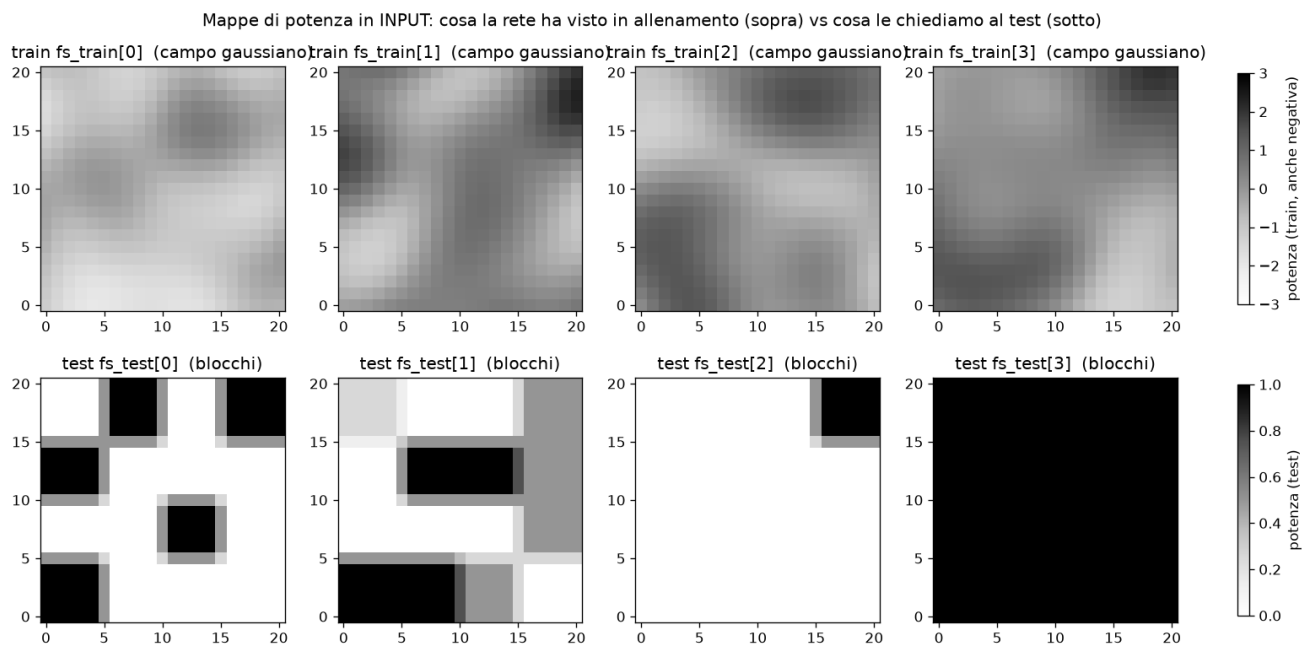


Fig. 4 — Sopra: 4 delle 10.000 mappe di training (campi gaussiani). Sotto: 4 delle 10 mappe di test (blocchi).

Esperimento 4 — Training set misto: gaussiani + blocchi

Abbiamo scritto un generatore di mappe a blocchi (`gen_block_maps.py`: 1–6 rettangoli casuali, potenza 0,25–2, bordi frazionari come nei test) e allenato su 5.000 gaussiani + 5.000 blocchi. Stesso modello, stessi iperparametri dell'Esperimento 3: cambia solo il dato.

	solo gaussiani (paper)	misto gaussiani + blocchi
rel. L2 medio test	1,96%	1,14% (-42%)
MAPE	0,043%	0,028%
errore sul picco (max_l1)	0,019	0,015
mappe migliorate	—	8 su 10

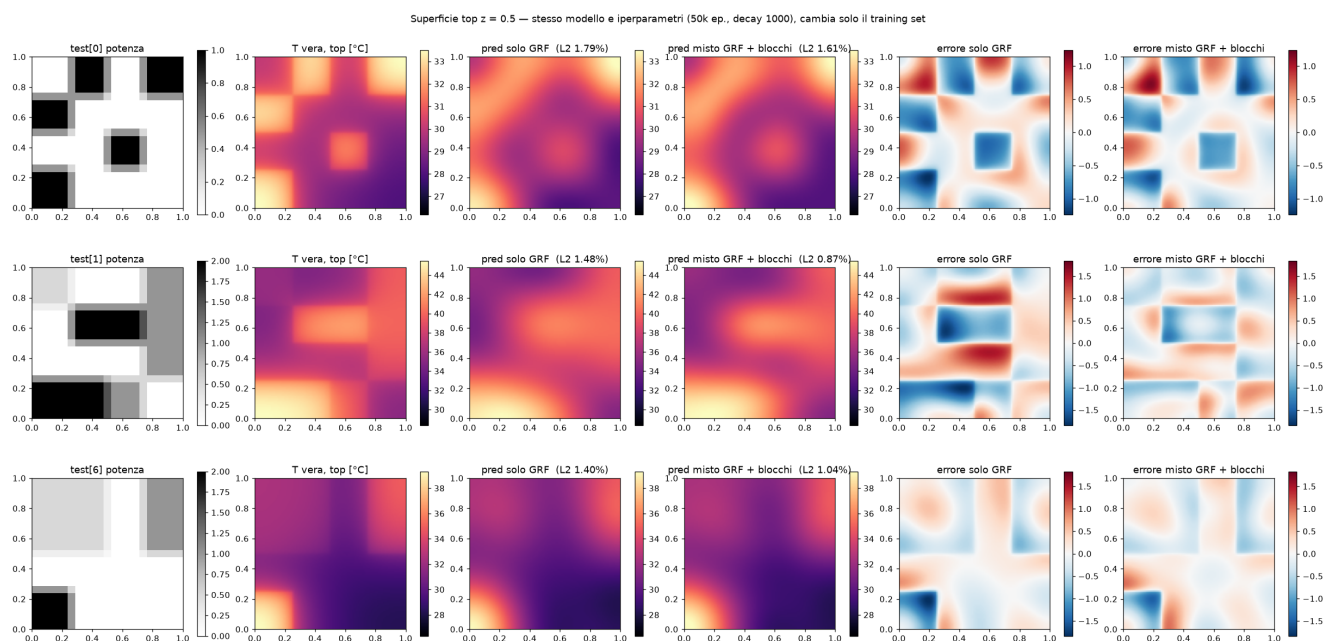


Fig. 5 — Tre mappe di test. Le bande d'errore lungo i bordi dei rettangoli si dimezzano con il training misto. I guadagni grossi sono sulle mappe a pochi blocchi netti: mappa 2 da 3,9% a 0,7%, mappa 5 da 4,2% a 0,9%.

Unica mappa che non migliora: la **numero 9**, che ha potenza massima 4 mentre il generatore arrivava a 2. Fuori distribuzione in ampiezza.

Esperimenti 5 e 6 — Solo blocchi? E potenza fino a 4?

Se i blocchi aiutano, perché non allenare *solo* su blocchi, con potenza estesa a 4? L'abbiamo provato (Esp. 5). Poi, per separare i due cambiamenti, anche il misto con potenza fino a 4 (Esp. 6).

training set	rel. L2 medio	max_l1	err. picco medio	mappa 9: err. picco
solo gaussiani (paper)	1,96%	0,019	0,48 °C	-2,79 °C
misto, potenza ≤ 2	1,14%	0,015	0,37 °C	-2,43 °C
misto, potenza ≤ 4	1,10%	0,011	0,27 °C	-1,34 °C
100% blocchi, potenza ≤ 4	4,42%	0,069	1,72 °C	-2,84 °C

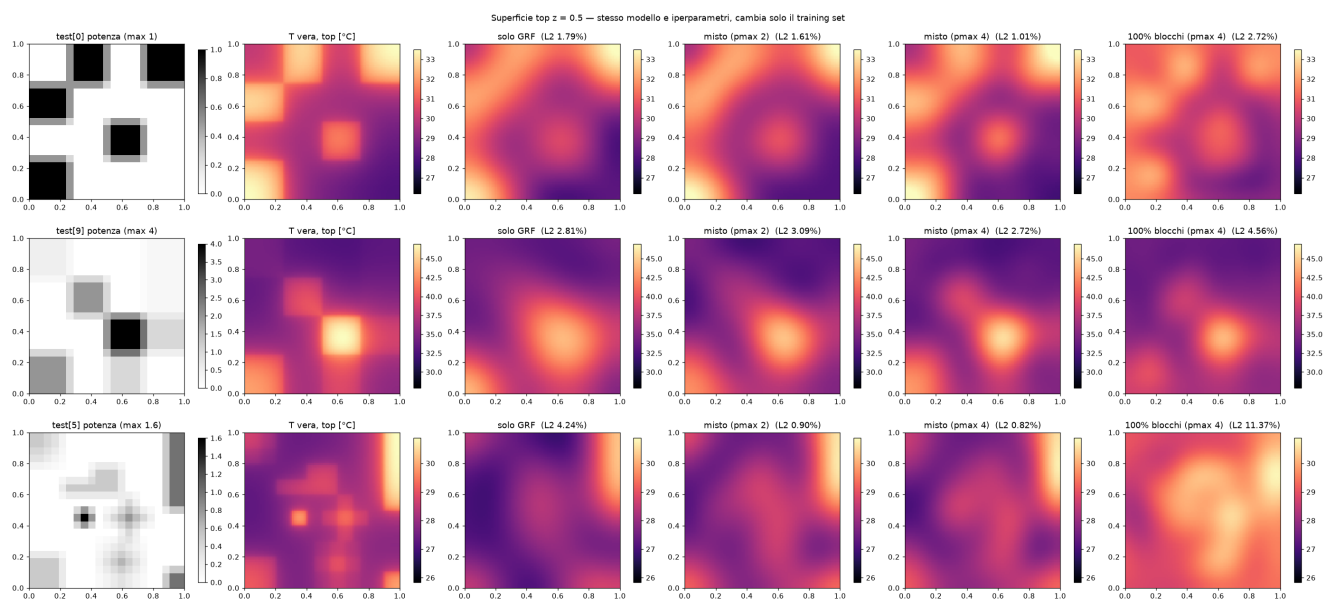


Fig. 6 — Stesso modello, quattro training set. Ultima colonna: il modello "solo blocchi" inventa una grossa macchia calda al centro della mappa 5 dove non c'è nulla. Penultima: il misto con potenza ≤ 4 mostra per la prima volta tutte e cinque le zone calde della mappa 0.

Sorpresa: 100% blocchi è il peggiore di tutti, peggio anche del paper, su tutte le 10 mappe. Non sfoca: *allucina*. Il confronto con l'Esp. 6 scagiona la potenza 4 (che anzi aiuta sui picchi): il colpevole è l'**assenza dei campi gaussiani**.

La spiegazione più probabile: l'equazione è lineare nella potenza, e i campi gaussiani — densi, con segni misti, senza struttura ripetuta — costringono la rete a imparare l'operatore vero. I blocchi da soli sono sparsi e "tutti uguali": la rete può memorizzare come appare la temperatura sotto un rettangolo senza imparare la fisica generale, e poi sbaglia appena la configurazione esce dalle statistiche viste. La scelta dei gaussiani nel paper, che sembrava strana, ha una ragione solida.

Cosa abbiamo imparato

- Il codice riproduce il paper (con due fix di compatibilità). Il default a 10.000 epoche interrompe il training mentre sta ancora imparando.
- Con lo scheduler originale il punto giusto è 20–25k epoche; con decadimento ogni 1.000 passi si sfruttano 50k.
- Il MAPE in Kelvin è una metrica generosa: nasconde errori di ~2 °C sui picchi, che sono la grandezza che conta.

- La sfocatura è in buona parte un problema di **dati**: aggiungere blocchi al training set taglia l'errore del 42% sulle stesse mappe di test del paper.
- Ma i campi gaussiani **servono**: senza, il modello allucina. Misto batte entrambi i puri.
- Il miglior modello: misto gaussiani + blocchi con potenza ≤ 4 , 50k epoche, decadimento ogni 1.000 passi. MAPE 0,031%, rel. L2 1,10%, errore medio sui picchi 0,27 °C (paper: 0,049%).

Cosa NON dimostrano questi esperimenti

- Il generatore di blocchi è stato disegnato *guardando* le 10 mappe di test. È una dimostrazione di meccanismo, non un confronto alla pari con il paper, il cui test era genuinamente fuori distribuzione.
- Non abbiamo un test set indipendente di blocchi con soluzione di riferimento: servirebbe un solutore numerico (quello del notebook richiede CUDA).
- Un solo seed per configurazione: differenze sotto lo 0,1% di rel. L2 (es. i due misti) sono rumore.
- Tutto sul caso "surface power". `heat_volumetric.py` ha ricevuto gli stessi fix ma non è stato eseguito (0,5 h su GPU nel paper).

Appendice — Riprodurre

Tutti i comandi dalla cartella `DeepOHeat-v1/`, con il venv attivo. Ogni run salva pesi, predizioni e metriche in `results/`.

```
python -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
# Esp. 1 — default (= paper)
python heat_surface.py
# Esp. 3 — decadimento lento
python heat_surface.py --epochs 50000 --decay_steps 1000
# Esp. 6 — miglior modello
python gen_block_maps.py --pmax 4 --seed 2 --out data/fs_train_surface_mixed_p4.npy
python heat_surface.py --epochs 50000 --decay_steps 1000 --train_data
data/fs_train_surface_mixed_p4.npy --tag _mixed_p4
```

File toccati: `heat_surface.py` (fix + 3 flag), `heat_volumetric.py` (fix), `gen_block_maps.py` (nuovo), `requirements.txt` (nuovo), `.gitignore` (+ `.venv`).